

Big Data: Offline Batch Processing

UNIVERSIDAD NACIONAL DE INGENIERÍA - FACULTAD DE INGENIERÍA ELÉCTRICA Y ELECTRÓNICA

CIB12(M). APRENDIZAJE MÁQUINA Y MINERÍA DE DATOS

Flores Quiliche, Fernando 20247516A <i>FIEE - Ing. de Telecomunicaciones</i> Lima, Perú f.flores.q@uni.pe	Pinto Acevedo, Kevin Eduardo 20220259H <i>FIEE - Ing. de Telecomunicaciones</i> Lima, Perú kevin.pinto.a@uni.pe	Valderrama Berrocal, Roberto Carlos 20222661H <i>FIEE - Ing. de Telecomunicaciones</i> Lima, Perú roberto.valderrama.b@uni.pe
Condori Musaja, Joge Luis 20221447B <i>FIEE - Ing. de Telecomunicaciones</i> Lima, Perú jorge.condori.m@uni.pe	Mego Gavidia, Eduardo Jeanpaul 20224080B <i>FIEE - Ing. de Telecomunicaciones</i> Lima, Perú eduardo.mego.g@uni.pe	Espinoza Tacuri, Jean Pierre Luis 20171198D <i>FIEE - Ing. Eléctrica</i> Lima, Perú jpespinoza@uni.pe

Abstract—El documento aborda el procesamiento de datos en entornos de Big Data mediante el enfoque de procesamiento por lotes offline, destacando su importancia para el análisis de grandes volúmenes de información histórica. Se describen tecnologías clave como HDFS para almacenamiento distribuido, Hive como sistema de data warehouse y Spark/SparkSQL como motores de procesamiento y análisis, resaltando su capacidad de escalabilidad, tolerancia a fallos y eficiencia. Además, se explica el modelo multicapa del data warehouse (ODS, DWD, DWS, ADS) y herramientas de ingesta como Sqoop y Loader, evidenciando un ecosistema integral para la gestión, transformación y explotación de datos a gran escala.

Index Terms—Big Data, procesamiento offline, batch processing, HDFS, Hive, Spark, SparkSQL, Data Warehouse, ETL, datos distribuidos.

CONTENTS

Introducción	2
I Concepto de Procesamiento por Offline Batch	2
I-A Solución de Procesamiento Offline	2
I-B Procesamiento por Lotes Offline (Offline Batch Processing)	2

Material elaborado por el Grupo 4 de la asignatura CIB12(M) - 2026-1

II HDFS: Almacenamiento de Datos (Data Storage)	3
II-A Marco Técnico del Procesamiento Offline	3
II-B Características de Rendimiento y Seguridad	3
II-C Operaciones Comunes en el Almacenamiento	4
II-D Limitaciones del Sistema	4
III Hive Data Warehouse: Conceptos Básicos de Hive	4
III-A ¿Qué es Hive?	4
III-B Características de Hive	4
III-C Arquitectura de Hive	4
III-D Modelo de almacenamiento de datos en Hive	4
III-E Diferencias entre tablas internas y externas	5
III-F Funciones integradas y UDF	5
III-G Optimización en Hive	5
IV Hive Data Warehouse: Desarrollo de HQL	5
IV-A HiveQL (HQL): Introducción	5
IV-B Ejemplo práctico: Sistema de información de empleados	6
IV-B1 Descripción del escenario	6

	IV-B2	Diseño de la tabla employees_info	6		VI-D2	El Dataset y el Optimizador Catalyst	11
	IV-B3	Creación de tablas (DDL)	6		VI-D3	Escenarios de Aplicación y Consultas	12
	IV-B4	Carga de datos (DML)	6				
	IV-B5	Consultas HQL (SELECT)	6	VII	SparkSQL: Análisis Offline - Desarrollo de SparkSQL		12
IV-C		Arquitectura Multi-capa del Data Warehouse	6	VIII	Herramientas de Recolección de Datos		14
V	Hive Data Warehouse: Modelo Multicapa		7	VIII-A	Intercambio de Datos con Sqoop		14
V-A		Hive como Sistema de Data Warehouse	7	VIII-B	Gestión Visual con Loader		14
V-B		Modelo Multicapa del Data Warehouse en Hive	7	VIII-C	Capacidades y Rendimiento Empresarial		14
V-C		Capas del Data Warehouse en Hive	7	IX	Conclusión		14
	V-C1	ODS – Operational Data Store	7	Conclusión			14
	V-C2	DWD – Data Warehouse Detail	7	References			15
	V-C3	DWS – Data Warehouse Service	7				
	V-C4	ADS – Application Data Store	7				
V-D		Ventajas del Modelo Multicapa en Hive	7				
V-E		Diferencia entre Data Warehouse y Data Mart	8				
VI	SparkSQL: Análisis Offline - Conceptos Clave		8				
	VI-A	Fundamentos y Escenarios de Apache Spark	8				
	VI-A1	Introducción a Apache Spark	8				
	VI-A2	Escenarios de Aplicación	8				
	VI-A3	Spark vs. MapReduce	8				
VI-B		El Modelo de Datos y Ejecución	8				
	VI-B1	RDD: Resilient Distributed Dataset	8				
	VI-B2	Shuffle y Dependencias	9				
	VI-B3	Etapas y el DAG	10				
	VI-B4	Transformaciones y Acciones	10				
VI-C		Contextos de Spark	10				
	VI-C1	SparkConf y SparkContext	10				
	VI-C2	SparkSession (Spark 2.0+)	11				
VI-D		SparkSQL y Datasets	11				
	VI-D1	Introducción a SparkSQL	11				

INTRODUCCIÓN

En la actualidad, el crecimiento exponencial de los datos ha impulsado el desarrollo de soluciones tecnológicas capaces de almacenar y procesar información a gran escala. En este contexto, el procesamiento por lotes offline surge como una alternativa fundamental para analizar datos históricos de manera eficiente, permitiendo a las organizaciones obtener conocimiento valioso para la toma de decisiones. Este enfoque se apoya en arquitecturas distribuidas y herramientas especializadas como HDFS, Hive y Spark, las cuales conforman un ecosistema robusto para el tratamiento de datos masivos dentro del ámbito del Big Data.

I. CONCEPTO DE PROCESAMIENTO POR OFFLINE BATCH

A. Solución de Procesamiento Offline

En el ecosistema actual de Big Data, las empresas se enfrentan a un crecimiento explosivo de datos que requieren no solo almacenamiento, sino un análisis profundo de la información histórica para la toma de decisiones estratégicas. Las soluciones tradicionales de procesamiento no logran escalar ante volúmenes de petabytes, lo que hace indispensable el uso de marcos de trabajo distribuidos.

B. Procesamiento por Lotes Offline (Offline Batch Processing)

El procesamiento por lotes offline se define fundamentalmente como el proceso de analizar y computar

volúmenes masivos de datos históricos almacenados previamente para generar resultados que serán utilizados por aplicaciones posteriores. Este método de procesamiento se caracteriza por los siguientes aspectos técnicos extraídos de los estándares de Huawei:

- **Baja urgencia temporal:** A diferencia de los sistemas de tiempo real, en el procesamiento offline la latencia de respuesta no es crítica; lo importante es la profundidad y el volumen del análisis
- **Baja urgencia temporal:** Escalabilidad masiva: Está diseñado para manejar datos a escala de Terabytes (TB) y Petabytes (PB)
- **Baja urgencia temporal:** Alto consumo de recursos: Debido a la magnitud de los datos, estas tareas ocupan una cantidad considerable de recursos de cómputo y almacenamiento durante su ejecución
- **Baja urgencia temporal:** Diversidad de formatos: Soporta el procesamiento de archivos estructurados, semiestructurados y no estructurados.

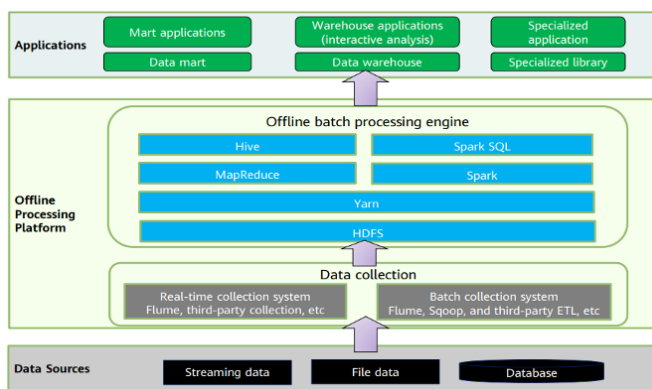


Fig. 1: Figura 1. Diagrama de flujo de datos del procesamiento por lotes offline. Fuente: Huawei Technologies Co., Ltd. (2022). HCIP-Big Data Developer V2.0 Training Material, Cap. 2, Sec. 2.1.

El procesamiento offline es la base de los almacenes de datos modernos (Data Warehouses) y se implementa típicamente mediante motores de ejecución como MapReduce, Spark o consultas en lenguaje HQL (Hive Query Language)

II. HDFS: ALMACENAMIENTO DE DATOS (DATA STORAGE)

A. Marco Técnico del Procesamiento Offline

El Sistema de Archivos Distribuidos de Hadoop (HDFS) constituye la piedra angular del almacenamiento en la solución de procesamiento offline. Fue diseñado originalmente bajo los principios del documento técnico

GFS (Google File System) y está optimizado para funcionar en hardware comercial común, asumiendo que las fallas de hardware son la norma y no la excepción. Arquitectura y Componentes Core HDFS utiliza una arquitectura de tipo maestro-esclavo que permite la gestión eficiente de metadatos y datos reales:

- **NameNode:** Es el nodo maestro que gestiona el espacio de nombres del sistema de archivos y mantiene los metadatos (nombres de archivos, permisos y la ubicación de los bloques de datos en el clúster).
- **DataNode:** Son los nodos esclavos que almacenan los bloques de datos reales. Son los encargados de ejecutar las operaciones de lectura y escritura solicitadas por los clientes.

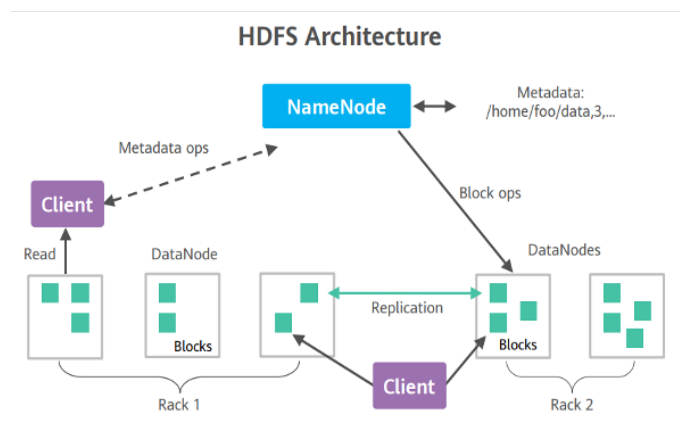


Fig. 2: Figura 2. Arquitectura del sistema HDFS (NameNode y DataNodes). Fuente: Huawei Technologies Co., Ltd. (2022). HCIP-Big Data Developer V2.0 Training Material, Cap. 2, Sec. 2.2.1.

B. Características de Rendimiento y Seguridad

Alta tolerancia a fallos: Los datos se replican automáticamente en diferentes nodos para asegurar que la información no se pierda si un servidor falla.

Acceso de alto rendimiento (Throughput): HDFS está optimizado para accesos secuenciales a grandes conjuntos de datos, lo que lo hace ideal para aplicaciones de procesamiento por lotes.

Mecanismo de Papelera (Recycle Bin): HDFS incluye una función de seguridad donde los archivos eliminados no se borran permanentemente de inmediato, sino que se mueven a una carpeta de reciclaje configurada en el archivo core-site.xml (parámetro fs.trash.interval) para permitir su recuperación rápida.

C. Operaciones Comunes en el Almacenamiento

El manejo de los datos en HDFS se realiza principalmente a través de comandos de consola (Shell Commands). Algunos de los más importantes son:

- **hdfs dfs -put:** Carga archivos desde el sistema local hacia el clúster de Big Data.
- **hdfs dfs -get:** Descarga archivos desde el clúster al sistema local.
- **hdfs dfs -ls:** Lista los archivos y directorios almacenados.
- **hdfs dfs -rm:** Elimina archivos (enviándolos a la papelera si está activa).

Type	Commands	Command Description
hdfs dfs	-cat	Displays the file content.
	-ls	Displays the directory list.
	-rm	Deletes a file.
	-put	Uploads the directory or file to HDFS.
	-get	Downloads directories or files to the local host from HDFS.
	-mkdir	Creates a directory.
	-chmod/-chown	Changes the group of the file.
	...	...
hdfs dfsadmin	-safemode	Security mode operation.
	-report	Reports service status.

Fig. 3: Figura 3. Comandos comunes de la interfaz de línea de comandos (Shell) para HDFS. Fuente: Huawei Technologies Co., Ltd. (2022). HCIP-Big Data Developer V2.0 Training Material, Cap. 2, Sec. 2.2.1

D. Limitaciones del Sistema

Es importante destacar que HDFS no es adecuado para ciertos escenarios, tales como:

- Almacenamiento de una gran cantidad de archivos pequeños (debido a la presión sobre la memoria del NameNode).
- Escrituras aleatorias o modificaciones de archivos ya existentes (HDFS es principalmente de "escribir una vez, leer muchas veces").
- Aplicaciones que requieren latencias de milisegundos.

III. HIVE DATA WAREHOUSE: CONCEPTOS BÁSICOS DE HIVE

A. ¿Qué es Hive?

Apache Hive es una herramienta de data warehouse diseñada para ejecutarse sobre Hadoop. Permite consultar y gestionar datos a escala de petabytes almacenados

en HDFS usando un lenguaje similar a SQL denominado HiveQL (HQL). Según la documentación oficial de Apache Hive [2], su arquitectura convierte las consultas HQL en trabajos distribuidos sobre MapReduce, Tez o Spark, permitiendo que analistas con conocimientos SQL trabajen con Big Data sin programar en MapReduce directamente. Tal como se describe en el material de entrenamiento oficial de Huawei [3] (sección 2.2.2, pág. 20), Hive actúa como una capa de abstracción sobre el ecosistema Hadoop para el procesamiento analítico de grandes volúmenes de datos.

B. Características de Hive

El material de entrenamiento Huawei HCIP [3] y la documentación oficial Apache [2] coinciden en las siguientes características clave:

- **ETL flexible y conveniente:** Facilita la extracción, transformación y carga de datos a gran escala.
- **Compatibilidad multi-motor:** Soporta MapReduce, Tez y Apache Spark como motores de cómputo.
- **Acceso directo a HDFS y HBase:** Sin necesidad de mover datos a una base de datos separada.
- **Fácil de usar y programar:** La similitud de HQL con SQL estándar reduce la curva de aprendizaje.

C. Arquitectura de Hive

Según la documentación oficial Apache Hive Wiki [4], la arquitectura consta de los siguientes componentes:

- **Interfaces de usuario (UI):** CLI (Command Line Interface), Web Interface y Thrift Server con soporte JDBC/ODBC.
- **Driver:** Núcleo del sistema. Contiene el Compiler (traduce HQL a plan de ejecución), el Optimizer (mejora el plan) y el Executor (envía el plan al motor de cómputo).
- **MetaStore:** Repositorio centralizado de metadatos (tablas, particiones, tipos de columnas). Se implementa sobre bases de datos relacionales como MySQL o Derby.

D. Modelo de almacenamiento de datos en Hive

Hive organiza los datos en una jerarquía definida en su documentación oficial [4]:

- **Database:** Espacio de nombres lógico que agrupa tablas relacionadas.
- **Tabla (Table):** Unidad fundamental de almacenamiento. Puede ser interna o externa.

- **Partición (Partition):** Subdivisión por valor de columna que optimiza las consultas al reducir datos escaneados.
- **Bucket (Cubo):** Subdivisión adicional usando función hash, mejora el rendimiento de JOINS y muestreo.

E. Diferencias entre tablas internas y externas

Esta distinción es fundamental. Según el Language-Manual DDL de Apache Hive [4] y el material Huawei [3]:

- **Tabla Interna (INTERNAL TABLE):** Al cargar, Hive mueve los archivos al directorio warehouse. Al ejecutar DROP TABLE se eliminan metadatos Y datos físicos.
- **Tabla Externa (EXTERNAL TABLE):** Al cargar, Hive solo registra la ubicación sin mover archivos. Al ejecutar DROP TABLE se eliminan solo los metadatos; los datos en HDFS permanecen intactos.

Comandos para gestionar el tipo de tabla [5]:

```
-- Consultar el tipo de tabla
DESC FORMATTED tableName;

-- Convertir tabla interna a externa
ALTER TABLE tableName SET
TBLPROPERTIES ('EXTERNAL'='TRUE');

-- Convertir tabla externa a interna
ALTER TABLE tableName SET
TBLPROPERTIES ('EXTERNAL'='FALSE');
```

F. Funciones integradas y UDF

El LanguageManual UDF de Apache Hive [5] documenta las funciones disponibles. Para explorarlas:

```
-- Lista todas las funciones disponibles
hive> SHOW FUNCTIONS;

-- Muestra el uso de una función
hive> DESC FUNCTION upper;

-- Muestra detalles extendidos
hive> DESC FUNCTION EXTENDED upper;
```

Categorías principales de funciones integradas [6]:

- **Funciones matemáticas:** round(), abs(), rand(), ceil(), floor().
- **Funciones de fecha:** to_date(), current_date(), date_diff(), date_add().
- **Funciones de cadena:** trim(), length(), substr(), upper(), lower(), concat().

Cuando las funciones nativas no cubren los requerimientos, se pueden desarrollar UDFs. Según la documentación oficial de Hive UDFs [7], existen tres tipos:

- **UDF (User-Defined Function):** Recibe una fila de datos y produce una fila de salida.
- **UDAF (User-Defined Aggregation Function):** Recibe múltiples filas y produce una única fila (similar a SUM, COUNT).
- **UDTF (User-Defined Table-Generating Function):** Recibe una fila y genera múltiples filas de salida.

Procedimiento de desarrollo de una UDF [7]:

- 1) Heredar de org.apache.hadoop.hive.ql.exec.UDF.
- 2) Implementar el método evaluate() con la lógica requerida.
- 3) Compilar, comprimir el JAR y subirlo a HDFS.
- 4) Crear función temporal en Hive con ADD JAR y CREATE TEMPORARY FUNCTION.
- 5) Invocar la función en las consultas HQL.

G. Optimización en Hive

El material Huawei HCIP [3] y la documentación Cloudera [8] describen las siguientes técnicas de optimización:

- **Agregación en Map:** SET hive.map.aggr=true; — Pre-agrega en fase Map, reduce tráfico de red.
- **Control de Data Skew:** SET hive.groupby.skewindata=true; — Genera 2 jobs MapReduce para balancear datos sesgados.
- **Map-side Join:** SET hive.auto.convert.join=true; — Carga tablas pequeñas en memoria, evita Reduce costoso.
- **Ejecución paralela:** SET hive.exec.parallel=true; — Fases independientes se ejecutan simultáneamente.
- **Máximo de threads paralelos:** SET hive.exec.parallel.thread.number=8;

IV. HIVE DATA WAREHOUSE: DESARROLLO DE HQL

A. HiveQL (HQL): Introducción

HiveQL es el lenguaje de consulta de Hive, diseñado con sintaxis similar al SQL estándar (ANSI SQL). Según el Tutorial oficial de Apache Hive [9], HQL soporta sentencias DDL (Data Definition Language) para crear y modificar estructuras, DML (Data Manipulation Language) para cargar e insertar datos, y sentencias SELECT para recuperar información. Al compilar, traduce las consultas en trabajos distribuidos, lo que lo hace adecuado para grandes volúmenes de datos.

El LanguageManual DML de Apache Hive [10] documenta en detalle todas las operaciones de manipulación de datos: LOAD DATA, INSERT, UPDATE y DELETE.

B. Ejemplo práctico: Sistema de información de empleados

El siguiente caso práctico, basado en el material de entrenamiento Huawei HCIP [3] (págs. 31-43), ilustra el ciclo completo de desarrollo HQL.

1) *Descripción del escenario:* Se analiza la información de empleados de una empresa. Se crean tres tablas: employees_info (información general), employees_contact (datos de contacto) y employees_info_extended (tabla particionada por año de ingreso).

2) Diseño de la tabla employees_info:

- **id (INT):** Número de empleado.
- **name (STRING):** Nombre completo.
- **usd_flag (STRING):** Moneda del salario — 'R' RMB (Yuan), 'D' USD.
- **salary (DOUBLE):** Monto del salario.
- **deductions (MAP<STRING, DOUBLE>):** Categoría impositiva, separador &
- **address (STRING):** Lugar de trabajo.
- **entrytime (STRING):** Año de ingreso.

3) *Creación de tablas (DDL):* Basado en LanguageManual DDL de Apache Hive [5] y el material Huawei [3]:

```
-- Tabla de información de empleados (EXTERNAL) OVERWRITE TABLE
CREATE EXTERNAL TABLE IF NOT EXISTS employees_info_extended
  id INT COMMENT 'No.',
  name STRING COMMENT 'Nombre',
  usd_flag STRING COMMENT 'R=RMB, D=USD',
  salary DOUBLE COMMENT 'Monto del salario',
  deductions MAP<STRING, DOUBLE> COMMENT 'Cat. impositiva',
  address STRING COMMENT 'Lugar de trabajo',
  entrytime STRING COMMENT 'Año de ingreso',
  ROW FORMAT DELIMITED
  FIELDS TERMINATED BY ','
  MAP KEYS TERMINATED BY '&'
  STORED AS TEXTFILE;
```

Tabla de contacto de empleados:

```
CREATE EXTERNAL TABLE IF NOT EXISTS employees_contact
  id INT COMMENT 'No.',
  tel_phone STRING COMMENT 'Número de teléfono',
  email STRING COMMENT 'Correo electrónico',
  ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
  STORED AS TEXTFILE;
```

Tabla extendida con particionamiento (PARTITIONED BY) [5]:

```
CREATE EXTERNAL TABLE IF NOT EXISTS employees_info
  id INT, name STRING, usd_flag STRING,
  salary DOUBLE, deductions MAP<STRING, DOUBLE>,
  address STRING, tel_phone STRING, email STRING,
  PARTITIONED BY (entrytime STRING)
  STORED AS TEXTFILE;
```

4) *Carga de datos (DML):* Según el LanguageManual DML de Apache Hive [9] y el material Huawei [3]:

```
-- Cargar desde sistema de archivos local
LOAD DATA LOCAL INPATH '/opt/hive_examples_data/employees_info.txt'
OVERWRITE INTO TABLE employees_info;
```

```
-- Cargar desde HDFS
LOAD DATA INPATH '/user/hive_examples_data/employees_contact.txt'
OVERWRITE INTO TABLE employees_contact;
```

5) *Consultas HQL (SELECT):* Consulta 1: Empleados con salario en USD mediante JOIN [9]:

```
SELECT a.name, b.tel_phone, b.email
FROM employees_info a JOIN
employees_contact b ON a.id = b.id
WHERE usd_flag = 'D';
```

Consulta 2: Insertar en tabla particionada los empleados ingresados en 2019 [5], [10]:

```
INSERT INTO employees_info
PARTITION (entrytime = '2019')
SELECT a.id, a.name, b.tel_phone
FROM employees_info a JOIN
employees_contact b ON a.id = b.id
WHERE a.entrytime = '2019';
```

Consulta 3: Conteo de registros y filtro por email [6]:

```
SELECT COUNT(*) FROM employees_contact;

SELECT a.name, b.tel_phone FROM
employees_info a
JOIN employees_contact b ON a.id = b.id
WHERE b.email LIKE '%cn';
```

C. Arquitectura Multi-capa del Data Warehouse

El material Huawei HCIP [2] (pág. 45) describe la arquitectura en capas que debe tener un Data Warehouse construido con Hive:

- **ODS (Operational Data Store):** Capa de datos crudos. Almacena los datos tal como llegan desde las fuentes operacionales.
- **DWD (Data Warehouse Detail):** Limpieza y estandarización. Misma granularidad que los datos originales.
- **DWS (Data Warehouse Service):** Agregación ligera basada en DWD. Facilita cálculos recurrentes.
- **ADS (Application Data Store):** Capa de servicio final. Proporciona datos procesados a las aplicaciones y reportes.

V. HIVE DATA WAREHOUSE: MODELO MULTICAPA

A. Hive como Sistema de Data Warehouse

Hive se utiliza como un sistema de Data Warehouse debido a su capacidad para procesar grandes volúmenes de datos estructurados y semiestructurados almacenados en HDFS. Permite analizar información proveniente de múltiples fuentes como logs de aplicaciones, bases de datos relacionales y sistemas distribuidos.

Una de sus principales características es el uso del enfoque schema-on-read, que permite definir la estructura de los datos en el momento de la consulta, brindando flexibilidad frente a cambios en las fuentes de datos. Además, al integrarse con otras herramientas del ecosistema Hadoop, Hive se convierte en una plataforma sólida para análisis de datos a nivel de terabytes y petabytes.

B. Modelo Multicapa del Data Warehouse en Hive

El modelo multicapa del Data Warehouse es una práctica recomendada para organizar los datos en entornos Big Data. Este modelo divide el flujo de datos en diferentes capas, cada una con una función clara, permitiendo un procesamiento ordenado y eficiente.

En Hive, el modelo multicapa se compone generalmente de cuatro capas principales: ODS, DWD, DWS y ADS. Esta estructura facilita la reutilización de datos, mejora el rendimiento de las consultas y asegura la calidad de la información utilizada.

C. Capas del Data Warehouse en Hive

1) **ODS – Operational Data Store:** La capa ODS representa el nivel inicial del Data Warehouse. En esta capa se almacenan los datos originales provenientes de los sistemas fuente, con mínima o ninguna transformación. Generalmente, los datos se almacenan por fecha, manteniendo la estructura original.

El objetivo principal del ODS es servir como respaldo histórico de los datos y como base para los procesos de

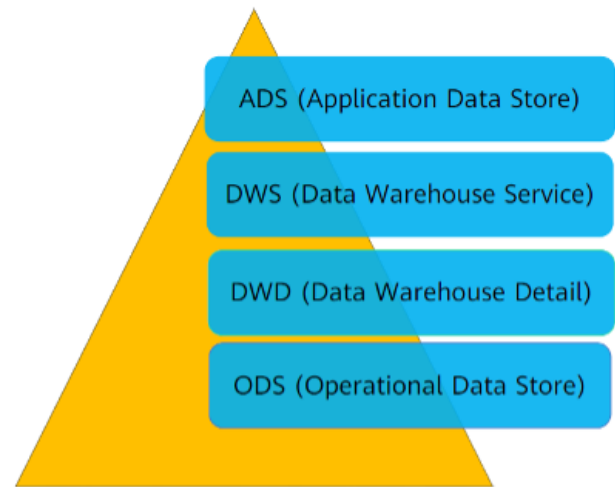


Fig. 4: Figura 4: “Hive Data Warehouse Layers: ODS – DWD – DWS – ADS”. Fuente: Huawei Technologies Co., Ltd. (2022), HCIP-Big Data Developer V2.0 Training Material)

transformación posteriores, permitiendo reprocesamientos en caso de errores.

2) **DWD – Data Warehouse Detail:** La capa DWD contiene los datos detallados y depurados. En esta etapa se aplican procesos de limpieza, eliminación de duplicados y estandarización de formatos, así como reglas básicas de negocio. Esta capa mantiene un alto nivel de granularidad, permitiendo realizar análisis más precisos y sirviendo como base para las agregaciones posteriores.

3) **DWS – Data Warehouse Service:** La capa DWS se encarga de realizar agregaciones ligeras sobre los datos detallados del DWD. En esta capa se calculan métricas e indicadores clave para el análisis del negocio, como totales, promedios y conteos. El uso de esta capa reduce la cantidad de cálculos repetitivos y mejora el rendimiento de las consultas analíticas.

4) **ADS – Application Data Store:** La capa ADS es el nivel final del Data Warehouse y está orientada al consumo por aplicaciones, dashboards y herramientas de Business Intelligence. Los datos están altamente agregados y estructurados de acuerdo con las necesidades del negocio.

D. Ventajas del Modelo Multicapa en Hive

El modelo multicapa ofrece múltiples ventajas, entre ellas:

- 1) Simplificación de procesos complejos

- 2) Reutilización de datos intermedios
- 3) Mejora de la calidad y consistencia de la información
- 4) Reducción de cálculos repetitivos
- 5) Mayor escalabilidad y facilidad de mantenimiento

Gracias a estas ventajas, este modelo es ampliamente utilizado en arquitecturas modernas de Big Data.

E. Diferencia entre Data Warehouse y Data Mart

Un Data Warehouse almacena información integrada de toda la organización, mientras que un Data Mart está enfocado en un área específica del negocio. En Hive, los Data Marts suelen construirse utilizando las capas superiores del Data Warehouse, especialmente DWS y ADS.

VI. SPARKSQL: ANÁLISIS OFFLINE - CONCEPTOS CLAVE

A. Fundamentos y Escenarios de Apache Spark

1) *Introducción a Apache Spark:* Apache Spark es un sistema de procesamiento por lotes distribuido basado en memoria, diseñado para superar las limitaciones de velocidad de los marcos de trabajo tradicionales como MapReducer. Surgió en AMPLab en la Universidad de California de Berkeley alrededor del año 2009 y fue liberado como proyecto de código abierto, convirtiéndose rápidamente en uno de los motores de procesamiento de datos a gran escala más populares de mundo.

La característica más distintiva de Spark es su capacidad de almacenar los resultados intermedios de las operaciones directamente en la RAM del clúster, en lugar de escribirlos en el disco duro como lo hace MapReducer. Esta estrategia elimina la latencia de las operaciones de I/O en disco, que contribuye el principal cuello de botella en el procesamiento de datos masivos.

Esta arquitectura hace a Spark ideal para la minería de datos y otros casos donde los mismos conjuntos de datos se procesan múltiples veces, como en el entrenamiento de modelos de aprendizaje automático o en los algoritmos de grafos iterativos.

2) *Escenarios de Aplicación:* La flexibilidad arquitectónica de Spark le permite abordar una amplia gama de escenarios de procesamiento de datos. Los tres principales son:

- **Procesamiento de Flujos (Streaming):** A través del componente Spark Streaming, es posible procesar flujos de datos en tiempo casi real. Los datos son divididos en pequeños lotes y procesados continuamente. Esto es útil en sistemas de detección

de fraude, monitoreo de redes sociales y análisis de logs en vivo.

- **Computación Iterativa:** Algoritmos que requieren pasar múltiples veces sobre los datos, como el algoritmo K-Means para clustering o el PageRank para analizar grafos, se benefician enormemente del almacenamiento en RAM. Spark puede retener los datos en memoria entre iteraciones, evitando la costosa lectura repetida desde HDFS.
- **Análisis de Consultas SQL:** Con el módulo Spark-SQL, los analistas de datos pueden ejecutar sentencias SQL estándar sobre conjuntos de datos distribuidos de escala masiva, procesando datos estructurados a nivel de Petabytes sin necesidad de conocer la programación en Scala o Java.

3) *Spark vs. MapReduce:* La comparación entre Apache Spark y el paradigma MapReduce de Hadoop es fundamental para entender la evolución del procesamiento de datos a gran escala. MapReduce, aunque revolucionario en su momento, sufre una limitación estructural crítica: cada operación de Map y Reduce escribe sus resultados intermedios en el disco HDFS, y la siguiente operación debe leerlos desde allí. Este patrón de escritura y lectura constante en disco lo vuelve extremadamente lento para tareas complejas.

Spark, en contraste, mantiene los datos en memoria a lo largo de toda la cadena de procesamiento, reescribiendo al disco solamente cuando es estrictamente necesario o cuando los datos no caben en RAM. Los benchmarks publicados demuestran que Spark puede ser hasta 100 veces más rápido que MapReduce en ciertos workloads, especialmente en los de tipo iterativo.

B. El Modelo de Datos y Ejecución

1) *RDD: Resilient Distributed Dataset:* El RDD o conjunto de datos distribuido resiliente es la estructura de datos fundamental y primitiva de Apache Spark. Fue introducida por (Zaharia, y otros, 2012) y representa la abstracción central sobre la que se construye todo el modelo de ejecución de Spark. Un RDD posee tres propiedades esenciales:

- **Inmutable y de solo lectura:** Una vez creado, un RDD no puede ser modificado. Las transformaciones generan nuevos RDDs derivados del original.
- **Particionado:** Los datos son divididos automáticamente en particiones distribuidas a través de los nodos del clúster, permitiendo el procesamiento en paralelo.

- **Resiliente:** Si un nodo del clúster falla y se pierden sus particiones, Spark puede recalcularlas automáticamente gracias al linaje, que es el registro de todas las transformaciones aplicadas para generar ese RDD.

Cuando Spark carga un conjunto de datos desde HDFS, S3, una base de datos relaciona u otra fuente, los representa internamienteoe como un RDD. Todas las operaciones posteriores de transformación y análisis se aplican sobre esta abstracción. El programador no necesita gestionar manualmente la distribución de datos no la tolerancia a fallos; Spark lo administra de forma transparente.

2) *Shuffle y Dependencias:* Las relaciones entre RDDs son categorizadas por Spark en dos tipos de dependencias, cuya naturaleza tiene un impacto directo en el rendimiento de la aplicación:

a) *Dependencia Estrecha:* En una dependencia estrecha, cada partición del RDD padre es utilizada por, como máximo, una sola partición del RDD hijo. Esto significa que la computación puede ejecutarse en paralelo en cada nodo de forma completamente independiente, sin necesidad de comunicación entre nodos. Son las operaciones más eficientes en Spark.

Narrow Dependencies:

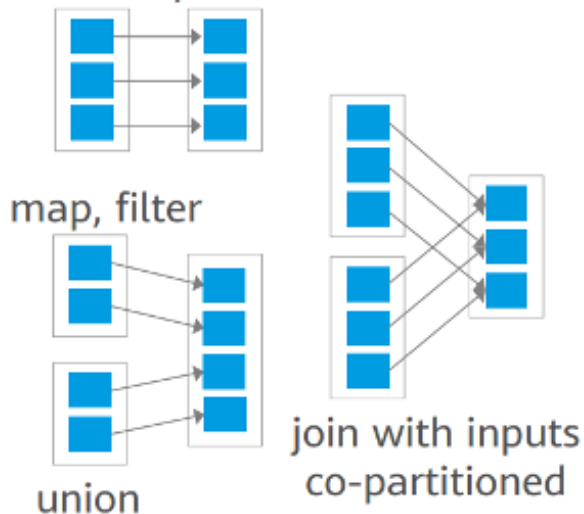


Fig. 5: Figura 5. Fuente: Huawei Technologies Co., Ltd. (2022). HCIP-Big Data Developer V2.0 Training Material, Cap. 2, Sec. 2.2.

Ejemplos típicos incluyen:

- map: aplica una función a cada elemento del RDD

- filter: selecciona elementos que cumplen una condición
- flatMap: Similar a map pero puede generar multiples elementos por entrada
- union: combina dos RDDs sin reorganizar datos

b) *Dependencia Ancha:* En una dependencia ancha, cada partición del RDD hijo puede depender de multiples particiones del RDD padre. Esto obliga a Spark a realizar una operación conocida como Shuffle: el intercambio y reorganización de datos a travez de la red entre todos los nodos del cluster. El shuffle es la operación más costosa en Spark, ya que involucra serialización de datos, trasferencia por red y escritura temporal en disco.

Wide Dependencies:

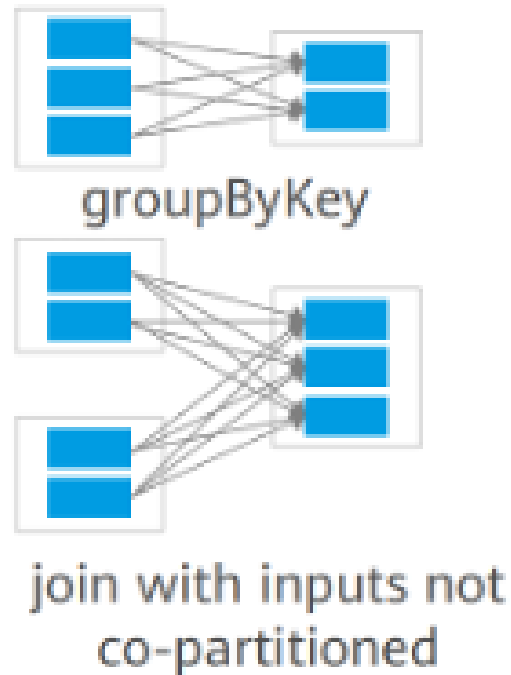


Fig. 6: Figura 6. Fuente: Huawei Technologies Co., Ltd. (2022). HCIP-Big Data Developer V2.0 Training Material, Cap. 2, Sec. 2.2.

Ejemplos incluyen:

- groupByKey: agrupa todos los valores con la misma clave
- reduceByKey: agrega valores por clave
- join: combina dos RDDs basándose en una clave común
- sortByKey: ordena los datos por clave

3) *Etapas y el DAG*: Cuando el programador define una serie de transformaciones sobre un RDD, Spark no las ejecuta inmediatamente. En cambio, construye un plan de ejecución representado como una DAG (Grafo Acíclico Dirigido), que es una representación visual y computacional de todas las operaciones que deben realizarse y sus dependencias.

El DAG Scheduler de Spark analiza este grafo y lo divide en etapas. El criterio de división es simple pero crucial: cada vez que el plan de ejecución encuentra una dependencia ancha, se crea un nuevo Stage. Todas las transformaciones con dependencias estrechas que no requieren Shuffle son agrupadas en el mismo Stage y ejecutadas en pipelines locales en cada nodo, lo que las hace muy eficientes.

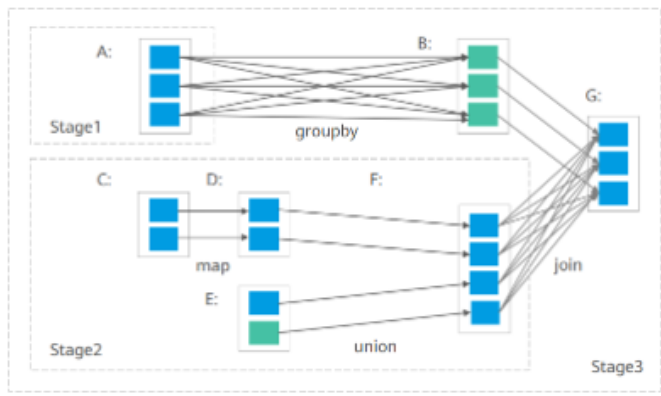


Fig. 7: Figura 7. Fuente: Huawei Technologies Co., Ltd. (2022). HCIP-Big Data Developer V2.0 Training Material, Cap. 2, Sec. 2.2.

4) *Transformaciones y Acciones*: Todas las operaciones en Spark se clasifican en dos categorías fundamentales que determinan cuando y como se ejecuta el cálculo:

a) *Transformaciones (Lazy Evaluation)*: Las transformaciones son operaciones perezosas (lazy): cuando se invoca una transformación sobre un RDD, Spark no ejecuta el cálculo de inmediato. En cambio, simplemente registra la operación en el DAG y retorna un nuevo RDD lógico. La ejecución real queda diferida hasta que sea estrictamente necesaria. Este diseño permite a Spark optimizar el plan de ejecución completo antes de comenzar cualquier cómputo real, aplicando optimizaciones como la fusión de operaciones y la eliminación de cálculos redundantes.

Ejemplos de transformaciones: `map()`, `filter()`, `groupByKey()`, `reduceByKey()`, `join()`, `distinct()`, `sample()`.

b) *Acciones*: Las acciones son las operaciones que disparan la ejecución real del plan registrado en el DAG. Cuando se invoca una acción, Spark envía el job completo al clúster para su procesamiento y retorna un resultado concreto al programa principal (driver), o escribe los datos en un almacenamiento externo. Cada llamada a una acción genera un nuevo Job en Spark.

Ejemplos de acciones: `collect()` (retorna todos los datos al driver), `count()` (cuenta elementos), `first()`, `take(n)`, `reduce()`, `saveAsTextFile()`, `foreach()`.

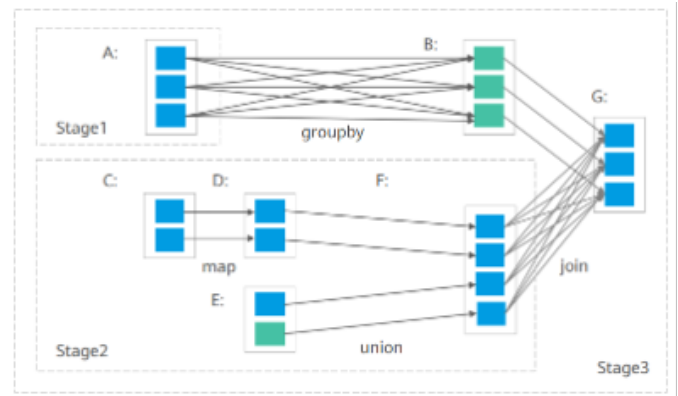


Fig. 8: Figura 7. Fuente: Huawei Technologies Co., Ltd. (2022). HCIP-Big Data Developer V2.0 Training Material, Cap. 2, Sec. 2.2.

C. Contextos de Spark

1) *SparkConf y SparkContext*: Para interactuar con un clúster de Spark, toda aplicación debe establecer una conexión y configurar los parámetros de ejecución. Esto se realiza a través de dos objetos fundamentales: SparkConf y SparkContext.

- **SparkConf**: Es el objeto de configuración de Spark. Permite especificar todos los parámetros de la aplicación, como el nombre de la aplicación, la URL del Master de clúster, la cantidad de memoria asignada a cada executor, el número de cores a utilizar, entre otros. Actúa como un contenedor de propiedades clave-valor.

```
val conf = new SparkConf()
    .setAppName("MiAplicacionSpark")
    .setMaster("spark://master:7077")
    .set("spark.executor.memory", "4g")
```

- **SparkContext**: Es el punto de entrada principal y la conexión real al clúster de Spark. A través de SparkContext, el programa driver se comunica con el Clúster Manager (YARN, Mesos o Standalone),

solicita recursos, y lanza las tareas de cómputo. Es el objeto que permite crear los primeros RDDs a partir de datos externos o colecciones locales.

```
val sc = new SparkContext(conf)
val rdd = sc.textFile("hdfs://namenode.
datos/archivo.txt")
val resultado =
rdd.filter(_.contains("ERROR")).count
```



Fig. 9: Figura 8. Fuente: Huawei Technologies Co., Ltd. (2022). HCIP-Big Data Developer V2.0 Training Material, Cap. 2, Sec. 2.2.

2) *SparkSession (Spark 2.0+)*: A partir de la versión 2.0 de Apache Spark, se introdujo un cambio arquitectónico significativo con la aparición del objeto SparkSession. Antes de esta versión, los desarrolladores debían gestionar múltiples puntos de entrada dependiendo de la funcionalidad que necesitaban: SparkContext para operaciones con RDDs, SQLContext para SparkSQL, HiveContext para integración con Hive, entre otros. Esta multiplicidad de contextos generaba confusión y código redundante.

SparkSession resuelve este problema al encapsular y unificar todos estos contextos bajo un único punto de entrada. Internamente, contiene un SparkContext y un SQLContext, pero el desarrollador solo necesita interactuar con el SparkSession, que delega las operaciones al contexto apropiado de forma transparente.

```
val spark = SparkSession.builder()
  .appName("MiAplicacion")
  .master("local[*]")
  .getOrCreate()
// Desde SparkSession se puede acceder
// al SparkContext:
val sc = spark.sparkContext
```

D. SparkSQL y Datasets

1) *Introducción a SparkSQL*: SparkSQL es el módulo de Apache Spark diseñado específicamente para el procesamiento de datos estructurados y semiestructurados. Su propuesta de valor central es permitir a los analistas y científicos de datos ejecutar sentencias SQL estándar directamente dentro de una aplicación Spark,

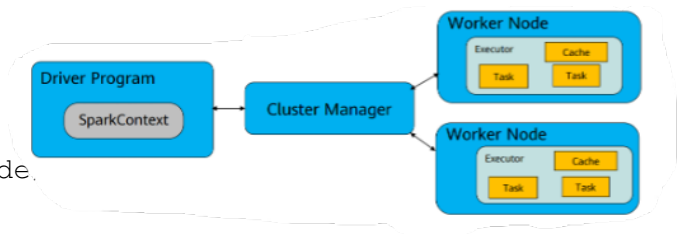


Fig. 10: Figura 9. Fuente: Huawei Technologies Co., Ltd. (2022). HCIP-Big Data Developer V2.0 Training Material, Cap. 2, Sec. 2.2.

sobre conjuntos de datos distribuidos en un clúster, sin necesidad de mover los datos a un servidor de base de datos relacional convencional.

A diferencia de los RDDs, que operan sobre datos sin esquema definido, SparkSQL trabaja con datos que tienen una estructura conocida (columnas y tipos de datos), lo que permite al motor de Spark realizar optimizaciones avanzadas durante la ejecución. SparkSQL integra procesamiento relacional con la capacidad de programación funcional de Spark, ofreciendo lo mejor de ambos mundos.

Adicionalmente, SparkSQL proporciona una capa de conectividad estándar mediante JDBC y ODBC, lo que permite que herramientas de Business Intelligence como Tableau, Power BI o Qlik se conecten directamente a Spark para realizar consultas analíticas.

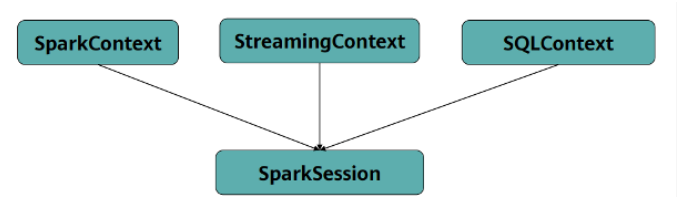


Fig. 11: Figura 10. Fuente: Huawei Technologies Co., Ltd. (2022). HCIP-Big Data Developer V2.0 Training Material, Cap. 2, Sec. 2.2.

2) *El Dataset y el Optimizador Catalyst*: El DataSet es la abstracción de datos de alto nivel ofrecida por SparkSQL. Representa una colección de objetos fuertemente tipados distribuida en el clúster. A diferencia del RDD genérico, un DataSet conoce el esquema de los datos que contiene. En Scala y Java, un DataSet es una colección tipada (Dataset[T]), mientras que en Python y R se usa el DataFrame, que es equivalente a un DataSet de filas genéricas (Dataset[Row]).

La característica más poderosa de SparkSQL es el

optimizador Catalyst. Cuando el programador escribe una consulta SQL o una cadena de operaciones sobre un DataFrame, Catalyst realiza el siguiente proceso de optimización en varias fases:

- **Análisis:** Verifica que la consulta sea semánticamente correcta (columnas existe, tipos son compatibles).
- **Plan lógico:** Construye una representación abstracta de lo que se quiere hacer sin considerar aún cómo hacerlo.
- **Optimización del Plan Lógico:** Aplica reglas de optimización algebraica, como el pushdown de filtros (mover los filtros lo más cerca posible de la fuente de datos para reducir el volumen procesado), eliminación de columnas no usadas y otras transformaciones equivalentes más eficientes.
- **Plan Físico:** Genera uno o varios planes de ejecución concretos y elige el más eficiente basándose en estadísticas del clúster y de los datos.
- **Generación de Código:** Produce código Java compilado (bytecode) optimizado para ejecutar el plan físico seleccionado, aprovechando las características del hardware moderno.

3) *Escenarios de Aplicación y Consultas:* SparkSQL esta especialmente optimizado para escenarios de análisis offline sobre volúmenes de datos a escala de Petabytes (PB), en situaciones donde no se requiere procesamiento en tiempo real estricto. Es el componente ideal para:

- Generación de reportes y dashboards a partir de warehouses de datos distribuidos en HDFS o almacenamiento cloud.
- Transformaciones ETL (Extract, transform, load) complejas sobre datos históricos.
- Análisis exploratorio de grandes volúmenes de datos estructurados por parte de equipos de ciencia de datos.
- Integración con herramientas de visualización mediante JDBC/ODBC.

A continuación, se presentan ejemplos conceptuales de las operaciones mas comunes en SparkSQL utilizando la API de DataFrame: Selección y proyección de columnas:

```
// Seleccionar columnas específicas
// de un DataFrame
val df = spark.read.parquet("hdfs:
///datos/ventas")
df.select("producto", "region",
"monto").show()
```

Filtrado de datos

```
// Filtrar registros que cumplan
// una condición
// Equivalente en SQL puro
df.filter(df("monto") > 1000).show()
df.createOrReplaceTempView("ventas")
spark.sql("SELECT * FROM ventas
WHERE monto > 1000").show()
```

Ordenamiento de Resultados

```
// Ordenar por una columna
// de forma descendente
df.sort(df("monto").desc).show(20)
```

Agrupación y Agregación

```
// Calcular el total de ventas por
// region df.groupBy("region")
//.agg(sum("monto").as("total_ventas"),
//count("*").as("num_transacciones"))
.sort(desc("total_ventas"))
.show()
```

La unificación de la API SQL declarativa con la potencia de un motor de procesamiento distribuido como Spark hace de SparkSQL una herramienta indispensable en cualquier arquitectura de Data Lake o Data Warehouse moderna, capaz de escalar horizontalmente al añadir nuevos nodos al clúster sin necesidad de modificar las consultas o el código de la aplicación.

VII. SPARKSQL: ANÁLISIS OFFLINE - DESARROLLO DE SPARKSQL

En continuidad con el desarrollo del presente capítulo sobre procesamiento batch offline, resulta pertinente examinar el papel de SparkSQL como componente orientado al tratamiento de datos estructurados dentro del entorno de Apache Spark. Luego de haber abordado los elementos generales del procesamiento offline y parte de su base técnica, se procede a analizar cómo este módulo se integra al marco de trabajo distribuido y de qué manera puede emplearse en el desarrollo de aplicaciones analíticas.

De acuerdo con Huawei (2022), SparkSQL es un módulo de Spark utilizado para procesar datos estructurados, el cual permite el uso de sentencias SQL dentro de aplicaciones Spark. Asimismo, dichas sentencias son transformadas en planes de ejecución sobre RDD y enviadas a SparkCore para su procesamiento, lo que permite articular el uso de SQL con la ejecución distribuida propia del entorno Spark.

SparkSQL dentro del marco de Spark Para comprender el funcionamiento de SparkSQL, es necesario situarlo dentro del contexto general de Spark. Según Huawei (2022), Spark es un sistema distribuido de procesamiento batch en memoria que divide las tareas y las asigna a múltiples CPU, almacenando los resultados intermedios en memoria. Este enfoque reduce las operaciones de entrada y salida sobre disco y contribuye a mejorar la eficiencia del procesamiento de datos.

Asimismo, Huawei (2022) indica que Spark presenta ventajas frente a MapReduce en aspectos como la menor latencia, la flexibilidad del modelo de programación y la tolerancia a fallos. En este contexto, SparkSQL se configura como una extensión especializada de Spark que permite aplicar estas capacidades al análisis de datos estructurados, manteniendo coherencia con el enfoque general del procesamiento distribuido.

Definición y alcance de SparkSQL en el análisis offline Dentro del análisis offline, SparkSQL se define como el módulo encargado del procesamiento de datos estructurados mediante consultas SQL en aplicaciones Spark (Huawei, 2022). Estas consultas son convertidas en planes de ejecución que se procesan en el entorno distribuido, lo que permite realizar análisis sobre grandes volúmenes de datos sin requerir mecanismos de consulta en tiempo real.

En este sentido, Huawei (2022) señala que SparkSQL es aplicable en escenarios donde no se requiere real-time query y que puede emplearse sobre volúmenes de datos a gran escala. Por ello, su uso se alinea con el enfoque batch del procesamiento offline desarrollado a lo largo del capítulo.

Conceptos básicos asociados al uso de SparkSQL Para comprender la implementación presentada, es necesario considerar algunos conceptos básicos. Huawei (2022) describe SparkSession como el punto de entrada unificado para utilizar las funciones de Spark, mientras que SparkConf permite establecer parámetros de ejecución y SparkContext representa la conexión con el clúster.

Asimismo, el material menciona el DataSet como un conjunto fuertemente tipado de objetos, asociado a un plan lógico y sujeto a evaluación diferida (lazy evaluation). Del mismo modo, se indica que las acciones son las operaciones que desencadenan la ejecución del procesamiento. Estos elementos constituyen el soporte conceptual necesario para comprender el desarrollo de una aplicación basada en SparkSQL (Huawei, 2022).

Escenario de desarrollo propuesto Huawei (2022) plantea un caso práctico centrado en el análisis de registros de visitas de compradores en línea. El objetivo consiste en obtener estadísticas de las usuarias que pasan más de dos horas realizando compras en línea. Para ello, se utiliza un archivo denominado log1.txt, cuyos registros contienen tres campos: nombre, género y tiempo de permanencia en minutos, separados por comas.

Este escenario corresponde a un caso de análisis offline, en tanto se trabaja con registros previamente almacenados y estructurados, a partir de los cuales se busca obtener información mediante operaciones de consulta y agregación (Huawei, 2022).

Procedimiento general de desarrollo El procedimiento descrito por Huawei (2022) sigue una secuencia definida. En primer lugar, se crea una tabla y se importan los archivos de log. En segundo lugar, se filtra la información correspondiente a las usuarias. Posteriormente, se resume el tiempo total que cada compradora pasa en línea. Finalmente, se seleccionan aquellos casos cuyo tiempo acumulado supera las dos horas.

Esta secuencia evidencia un proceso estructurado de análisis que comprende la preparación de los datos, la selección de la información relevante, la agregación de resultados y la obtención del resultado final, en coherencia con el enfoque del procesamiento batch offline.

Implementación del caso práctico en Scala La implementación del caso se desarrolla en Scala. Según Huawei (2022), el código define una case class denominada FemaleInfo, con los campos name, gender y stayTime, para representar los registros del archivo. A continuación, la aplicación configura su nombre mediante SparkConf, crea una SparkSession, obtiene el SparkContext y lee el archivo mediante sc.textFile(args(0)).

Posteriormente, cada línea es dividida, convertida al tipo correspondiente y transformada en un DataFrame, el cual es registrado como una tabla temporal denominada FemaleInfoTable. Esta transformación permite que los datos puedan ser consultados mediante sentencias SQL dentro de la aplicación Spark (Huawei, 2022).

Consulta y obtención del resultado analítico Una vez registrada la tabla temporal, se ejecuta una consulta SQL que selecciona el nombre y calcula la suma del tiempo de permanencia para los registros cuyo género es femenino, agrupando por nombre (Huawei, 2022). Posteriormente, se aplica un filtro con la condición stayTime $\geq$ 120, y los resultados son recuperados mediante la operación

collect().

Es importante precisar que, aunque el objetivo del caso menciona usuarias que pasan “más de 2 horas”, el criterio implementado en el código corresponde a $t = 120$ minutos. Por lo tanto, ambas formulaciones deben distinguirse para mantener fidelidad a la fuente (Huawei, 2022).

Ejecución de la aplicación en entorno distribuido Finalmente, Huawei (2022) indica que la aplicación debe empaquetarse en un archivo JAR, cargarse en un entorno Linux y ejecutarse mediante spark-submit en modo yarn-cluster. El comando especifica la clase principal y la ruta del archivo de entrada, lo que permite ejecutar la aplicación en un clúster distribuido.

Este proceso evidencia que el análisis desarrollado no se limita a la formulación de consultas, sino que forma parte de un flujo completo de desarrollo y ejecución dentro de un entorno distribuido, en coherencia con el enfoque del procesamiento offline.

VIII. HERRAMIENTAS DE RECOLECCIÓN DE DATOS

A. Intercambio de Datos con Sqoop

Sqoop es una herramienta de código abierto fundamental para transferir datos entre el ecosistema Hadoop (Hive) y bases de datos relacionales tradicionales como MySQL, Oracle y PostgreSQL. Su funcionamiento principal permite tanto la importación de datos hacia el sistema de archivos HDFS como la exportación desde HDFS hacia bases de datos relacionales. Para lograr eficiencia, Sqoop utiliza parámetros como split-by para dividir los datos en regiones que son procesadas en paralelo por diferentes tareas de mapeo.

B. Gestión Visual con Loader

Loader es una herramienta de carga de datos que facilita el intercambio de información entre sistemas de archivos y bases de datos relacionales. A diferencia de otras herramientas, ofrece una interfaz gráfica basada en un asistente visual para la configuración de trabajos, lo que simplifica la operación para el usuario. Además, permite la limpieza y conversión de datos durante el proceso de recolección y admite la programación de tareas periódicas para una automatización completa.

C. Capacidades y Rendimiento Empresarial

Las soluciones de recolección, como las optimizaciones hechas por Huawei en Loader, se centran en cuatro características clave para entornos empresariales: Alto rendimiento: Utiliza MapReduce para procesar

datos de forma paralela. Confiabilidad: Los servidores se despliegan en modo activo/espera y los trabajos pueden reintentarse tras un fallo sin dejar datos residuales. Seguridad: Implementa autenticación Kerberos y gestión de derechos de usuario. Versatilidad: Es capaz de recolectar datos desde diversas fuentes, como servidores SFTP o bases de datos de servicio, para almacenarlos de forma organizada en HDFS.

IX. CONCLUSIÓN

Valderrama Berrocal, Roberto Carlos

Desde mi perspectiva como estudiante, el estudio del procesamiento por lotes offline y la arquitectura HDFS me permite comprender que el Big Data no se trata solo de la velocidad, sino de la capacidad de persistir y estructurar la memoria histórica de una organización. HDFS es el cimiento invisible que sostiene todo el ecosistema; sin su capacidad de tolerancia a fallos y su diseño basado en el procesamiento distribuido, sería imposible realizar los análisis complejos que hoy ejecutan motores como Hive o Spark. Entender que el hardware fallará y diseñar un sistema que sea resiliente a ello es, a mi parecer, la lección de ingeniería más valiosa de este capítulo.

Pinto Acevedo, Kevin Eduardo

El estudio de Apache Hive como componente central del ecosistema de Big Data me ha permitido comprender cómo se estructura el procesamiento analítico a gran escala sobre plataformas distribuidas. Hive no es simplemente un motor de consultas: es una solución arquitectónica que democratiza el acceso al Big Data al permitir que analistas con conocimientos en SQL analicen volúmenes de datos en escala de petabytes sin necesidad de programar en MapReduce directamente. La claridad con la que la documentación oficial de Apache explica esta arquitectura demuestra la madurez del proyecto después de más de 18 años de desarrollo.

Uno de los aprendizajes más significativos ha sido comprender la diferencia entre tablas internas y externas, ya que esta distinción tiene implicaciones directas en la gestión del ciclo de vida de los datos en una organización. Como lo documenta el LanguageManual DDL de Apache, un DROP TABLE en una tabla interna elimina los datos físicos de forma irreversible, mientras que en una tabla externa solo se eliminan los metadatos. Esta diferencia, aunque parezca sutil, puede marcar la pérdida total de datos en un entorno productivo si no se diseña con cuidado.

Finalmente, el desarrollo del caso práctico con HQL - desde la definición del esquema con tipos complejos como MAP STRING, DOUBLE, pasando por la carga de datos y el particionamiento, hasta la ejecución de consultas analíticas con JOIN - me brindó una visión completa del ciclo de vida de un proyecto de data warehouse real. Las técnicas de optimización documentadas por Cloudera como el map-side join y la ejecución paralela son habilidades esenciales que marcan la diferencia entre una implementación funcional y una solución eficiente y escalable en producción.

Espinoza Tacuri, Jean Pierre Luis

En conclusión, Apache Hive junto con el modelo multicapa del Data Warehouse constituye una solución eficiente para el manejo de grandes volúmenes de datos en entornos Big Data. La separación en capas como ODS, DWD, DWS y ADS permite mejorar la organización, rendimiento y calidad de los datos, facilitando una toma de decisiones informada y confiable.

Condori Musaja, Joge Luis

Se concluye que gracias al módulo SparkSQL y al optimizador Catalyst, el análisis de datos a escala de Petabytes ya no es exclusivo de programadores expertos en Scala o Java. Catalyst permite que consultas SQL estándar alcancen niveles de eficiencia similares o superiores al código de bajo nivel, facilitando que analistas utilicen herramientas de Business Intelligence sobre volúmenes masivos de datos.

También se concluye sobre la versatilidad y escalabilidad de de Spark, que se consolida como una plataforma unificada tras la introducción de SparkSession la cual puede manejar desde procesamiento por lotes y streaming hasta análisis relacional complejo, con una escalabilidad horizontal que permite aumentar la capacidad simplemente añadiendo nodos al clúster.

Mego Gavidia, Eduardo Jeanpaul

El análisis desarrollado permite situar a SparkSQL como un componente funcional dentro del procesamiento batch offline, cuya relevancia radica en su capacidad para operar datos estructurados mediante consultas SQL integradas en el entorno distribuido de Spark. A partir de lo expuesto por Huawei (2022), se observa que este módulo no solo proporciona un mecanismo de consulta, sino que se articula con la infraestructura de ejecución de Spark para transformar instrucciones declarativas en procesos

distribuidos sobre grandes volúmenes de datos.

El caso práctico presentado refuerza esta idea al mostrar, de manera concreta, cómo un conjunto de registros puede ser estructurado, consultado y transformado mediante operaciones de filtrado y agregación, siguiendo una secuencia coherente con el enfoque del análisis offline. En este sentido, el ejemplo no se limita a ilustrar el uso de sentencias SQL, sino que evidencia el proceso completo de desarrollo de una aplicación analítica dentro del ecosistema de Spark.

De esta manera, el tratamiento de SparkSQL permite comprender su papel como puente entre el modelo declarativo de consulta y la ejecución distribuida propia del procesamiento batch, consolidándose como una herramienta clave para el análisis de datos estructurados en escenarios offline.

Flores Quiliche, Fernando

Las herramientas de recolección de datos como Sqoop y Loader representan la base operativa para la ingesta de información en ecosistemas de Big Data al permitir la transferencia fluida entre bases de datos relacionales y HDFS, destacando por su capacidad de procesamiento paralelo para el alto rendimiento, la facilidad de uso mediante asistentes visuales de configuración y la implementación de estándares de seguridad Kerberos garantizando que los datos recolectados de múltiples fuentes sean almacenados y organizados de forma óptima para su posterior análisis y toma de decisiones empresariales.

BIBLIOGRAFÍA

REFERENCES

- [1] Huawei Technologies Co., Ltd. (2022). HCIP-Big Data Developer V2.0 Training Material. Capítulo 2: Scenario-based Big Data Solutions - Offline Batch Processing.
- [2] Apache Software Foundation (2024). Apache Hive – Official website and documentation overview. <https://hive.apache.org/>
- [3] Huawei Technologies Co., Ltd. (2023). HCIP-Big Data Developer V2.0 Training Material. Capítulo 2: Big Data Scenario-based Solution – Offline Batch Processing (pp. 19-45). Huawei Learning.
- [4] Apache Software Foundation (2024). Apache Hive Wiki – Home. Architecture, MetaStore and storage model. <https://cwiki.apache.org/confluence/display/Hive/Home>
- [5] Apache Software Foundation (2024). LanguageManual DDL – Data Definition Statements. Apache Hive Wiki. <https://cwiki.apache.org/confluence/display/Hive/LanguageManual+DDL>
- [6] Apache Software Foundation (2024). LanguageManual UDF – Operators and User-Defined Functions. Apache Hive Wiki. <https://cwiki.apache.org/confluence/display/Hive/LanguageManual+UDF>

- [7] Apache Software Foundation (2024). Hive UDFs – User-Defined Functions guide. Apache Hive Wiki. <https://cwiki.apache.org/confluence/display/Hive/Hive+UDFs>
- [8] Cloudera (2024). Using Apache Hive – Cloudera Runtime 7.2.18. <https://docs.cloudera.com/runtime/7.2.18/using-hiveql/index.html>
- [9] Apache Software Foundation (2024). Apache Hive Tutorial. Apache Hive Wiki. <https://cwiki.apache.org/confluence/display/Hive/Tutorial>
- [10] Apache Software Foundation (2024). LanguageManual DML – Data Manipulation Statements. Apache Hive Wiki. <https://cwiki.apache.org/confluence/display/Hive/LanguageManual+DML>
- [11] Apache Software Foundation (2024). Apache Hadoop 3.4.1 – Official Documentation. <https://hadoop.apache.org/docs/current/>
- [12] Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M. J., Ghodsi, A., Gonzalez, J., Shenker, S., & Stoica, I. (2016). Apache Spark: A unified engine for big data processing. *Communications of the ACM*, 59(11), 56–65. <https://doi.org/10.1145/2934664>
- [13] White, T. (2015). *Hadoop: The definitive guide* (4th ed.). Sebastopol, CA: O’Reilly Media. <https://www.oreilly.com/library/view/hadoop-the-definitive/9781491901687/>
- [14] Kimball, R., & Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3rd ed.). Hoboken, NJ: John Wiley & Sons. <https://onlinelibrary.wiley.com/doi/book/10.1002/9781118530801>
- [15] Inmon, W. H. (2005). *Building the data warehouse* (4th ed.). Hoboken, NJ: John Wiley & Sons. <https://onlinelibrary.wiley.com/doi/book/10.1002/0471729765>
- [16] Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauley, M., Stoica, I. (2012). Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. *Proceedings of the USENIX NSDI 2012*, 15–28. [?]
- [17] Shanahan, J. G., & Dai, L. (2015). Large scale distributed data science using Apache Spark. *Proceedings of the 21st ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2323–2324. <https://doi.org/10.1145/2783258.2789993>